

DS Lab Assignment - 2: Odd 2026

Vaibhav Sharma (2501030279)

August 1, 2026

All source files here: [Github Repository](#)

Answer 1

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 class Node {
4     public:
5     int data;
6     Node* next;
7     Node(int value) {
8         data = value;
9         next = nullptr;
10    }
11 };
12 class LinkedList {
13     private:
14     Node* head;
15     public:
16     LinkedList() {
17         head = nullptr;
18     }
19     void insertHead(int value) {
20         Node* newNode = new Node(value);
21         if (head==NULL){
22             head=newNode;
23         }
24         else{
25             Node* curr=head;
26             while(curr->next!=NULL){
27                 curr=curr->next;
28             }
29             curr->next=newNode;
30         }
31     }
32     void cnt() {
33         Node * temp=head;
34         float ans=0;
35         int c=0;
36         while(temp!=NULL){
37             c++;
38             ans+=temp->data;
39             temp=temp->next;
40         }
41         cout<<"Total number of nodes are:"<<c<<" and the average
42             is:"<<fixed<<setprecision(2)<<ans/c<<"\n";
43     }
44     void last(int x) {
45         Node * temp=head;
46         int c=0;
47         while(temp!=NULL){
48             c++;
49             temp=temp->next;
```

```

49     }
50     temp=head;
51     for (int i=0;i<c-x;i++){
52         temp=temp->next;
53     }
54     while(temp!=NULL){
55         cout<<temp->data<<" ";
56         temp=temp->next;
57     }
58     cout<<"\n";
59 }
60 void middle() {
61     Node* fast=head;
62     Node* slow=head;
63     if(head==NULL){
64         return;
65     }
66     while(fast!=NULL && fast->next!=NULL){
67         fast=fast->next->next;
68         slow=slow->next;
69     }
70     if(slow->data%2==0) {
71         cout<<"The middle element "<<slow->data<<" is even"<<
72             "\n";
73     }
74     else {
75         cout<<"The middle element "<<slow->data<<" is odd"<<"
76             "\n";
77     }
78 }
79 void insertN(int n) {
80     int value;
81     for (int i = 0; i < n; i++) {
82         cout << "Enter data " << i + 1 << ": ";
83         cin >> value;
84         insertHead(value);
85     }
86 }
87 void display(int m) {
88     Node* temp = head;
89     for (int i=0;i<m;i++){
90         cout<<temp->data<<" ";
91         temp=temp->next;
92     }
93     cout<<"\n";
94 }
95 void deleteNode(int info) {
96     if (head == nullptr) {
97         cout << "No such value exists\n";
98         return;
99     }
100     if (head->data == info) {
101         Node* temp = head;
102         head = head->next;
103         delete temp;
104         return;
105     }
106     Node* curr = head;
107     Node* temp = head->next;
108     while (temp != nullptr) {
109         if (temp->data == info) {
110             curr->next = temp->next;
111             delete temp;
112             return;
113         }
114         curr = temp;
115         temp = temp->next;
116     }
117     cout << "No such value exists\n";
118 }

```

```

117 void interchangePairs(int a1, int a2, int b1, int b2) {
118     if (head == NULL || head->next == NULL)
119         return;
120     Node *prev1 = NULL, *first1 = NULL;
121     Node *prev2 = NULL, *first2 = NULL;
122     Node *prev = NULL;
123     Node *curr = head;
124     while (curr && curr->next) {
125         if (curr->data == a1 && curr->next->data == a2) {
126             prev1 = prev;
127             first1 = curr;
128             break;
129         }
130         prev = curr;
131         curr = curr->next;
132     }
133     prev = NULL;
134     curr = head;
135     while (curr && curr->next) {
136         if (curr->data == b1 && curr->next->data == b2) {
137             prev2 = prev;
138             first2 = curr;
139             break;
140         }
141         prev = curr;
142         curr = curr->next;
143     }
144     if (first1 == NULL || first2 == NULL)
145         return;
146     if (first1 == first2)
147         return;
148     Node *second1 = first1->next;
149     Node *after1 = second1->next;
150     Node *second2 = first2->next;
151     Node *after2 = second2->next;
152     if (after1 == first2) {
153         if (prev1)
154             prev1->next = first2;
155         else
156             head = first2;
157         second2->next = first1;
158         second1->next = after2;
159         first1->next = second2;
160     }
161     else if (after2 == first1) {
162         if (prev2)
163             prev2->next = first1;
164         else
165             head = first1;
166         second1->next = first2;
167         second2->next = after1;
168         first2->next = second1;
169     }
170     else {
171         if (prev1)
172             prev1->next = first2;
173         else
174             head = first2;
175         if (prev2)
176             prev2->next = first1;
177         else
178             head = first1;
179         second2->next = after1;
180         second1->next = after2;
181         swap(first1->next, first2->next);
182     }
183 }
184 };
185 int main() {
186     LinkedList list;
187     int n;

```

```

188     cout << "Enter number of nodes: ";
189     cin >> n;
190     list.insertN(n);
191     list.cnt();
192     cout<<"Enter number of nodes to display: ";
193     int m;
194     cin>>m;
195     list.display(m);
196     list.middle();
197     int x;
198     cout<<"Enter the node number from last: ";
199     cin>>x;
200     list.last(x);
201     int data;
202     cout<<"Enter the data to delete";
203     cin>>data;
204     list.deleteNode(data);
205     cout<<"The list after deletion:\n";
206     list.display(4);
207     list.interchangePairs(2,3,4,5);
208     return 0;
209 }

```

Answer 2

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4  struct Node {
5      string data;
6      Node* next;
7  };
8  class LinkedList {
9      private:
10     Node* head;
11     public:
12     LinkedList() {
13         head = NULL;
14     }
15     void insert(string value) {
16         Node* newNode = new Node;
17         newNode->data = value;
18         newNode->next = NULL;
19         if (head == NULL) {
20             head = newNode;
21             return;
22         }
23         Node* curr = head;
24         while (curr->next != NULL)
25             curr = curr->next;
26         curr->next = newNode;
27     }
28     void printList() {
29         if (head == NULL) {
30             cout << "List is empty.\n";
31             return;
32         }
33         Node* curr = head;
34         while (curr != NULL) {
35             cout << curr->data << " ";
36             curr = curr->next;
37         }
38         cout << endl;
39     }
40     void printStartingWith(char ch) {
41         Node* curr = head;
42         bool found = false;
43         while (curr != NULL) {
44             if (!curr->data.empty() && curr->data[0] == ch) {

```

```

45         cout << curr->data << endl;
46         found = true;
47     }
48     curr = curr->next;
49 }
50 if (!found)
51     cout << "No string starts with '" << ch << "'." <<
        endl;
52 }
53 void searchString(string key) {
54     Node* curr = head;
55     while (curr != NULL) {
56         if (curr->data == key) {
57             cout << "\"" << key << "\" exists in the linked
                list.\n";
58             return;
59         }
60         curr = curr->next;
61     }
62     cout << "\"" << key << "\" does not exist in the linked
        list.\n";
63 }
64 void longestString() {
65     if (head == NULL) {
66         cout << "List is empty.\n";
67         return;
68     }
69     Node* curr = head;
70     string longest = curr->data;
71     while (curr != NULL) {
72         if (curr->data.length() > longest.length())
73             longest = curr->data;
74         curr = curr->next;
75     }
76     cout << "Longest string: " << longest
77         << " (Length = " << longest.length() << ")" << endl;
78 }
79 void containsXYZ() {
80     Node* curr = head;
81     bool found = false;
82     while (curr != NULL) {
83         if (curr->data.find("xyz") != string::npos) {
84             cout << "\"" << curr->data
85                 << "\" contains \"xyz\".\n";
86             found = true;
87         }
88         curr = curr->next;
89     }
90     if (!found)
91         cout << "No string contains \"xyz\".\n";
92 }
93 void swapPositions(int p1, int p2) {
94     if (p1 == p2) {
95         cout << "Both positions are the same.\n";
96         return;
97     }
98     Node *node1 = NULL, *node2 = NULL;
99     Node* curr = head;
100     int pos = 1;
101     while (curr != NULL) {
102         if (pos == p1)
103             node1 = curr;
104         if (pos == p2)
105             node2 = curr;
106         curr = curr->next;
107         pos++;
108     }
109     if (node1 == NULL || node2 == NULL) {
110         cout << "Error: One or both positions do not exist.\n";
111         return;
112     }

```

```

111         return;
112     }
113     swap(node1->data, node2->data);
114     cout << "Strings interchanged successfully.\n";
115 }
116 };
117 int main() {
118     LinkedList list;
119     list.insert("apple");
120     list.insert("banana");
121     list.insert("xyzabc");
122     list.insert("orange");
123     list.insert("grapexyz");
124     list.insert("mango");
125     list.insert("apricot");
126     cout << "Original Linked List:\n";
127     list.printList();
128     cout << "\n(a) Print all nodes:\n";
129     list.printList();
130     cout << "\n(b) Strings starting with 'a':\n";
131     list.printStartingWith('a');
132     cout << "\n(c) Search for \"orange\":\n";
133     list.searchString("orange");
134     cout << "\nSearch for \"kiwi\":\n";
135     list.searchString("kiwi");
136     cout << "\n(d) Longest string:\n";
137     list.longestString();
138     cout << "\n(e) Strings containing \"xyz\":\n";
139     list.containsXYZ();
140     cout << "\n(f) Swap positions 2 and 6:\n";
141     list.swapPositions(2, 6);
142     cout << "Updated Linked List:\n";
143     list.printList();
144     cout << "\nAttempting to swap positions 7 and 10:\n";
145     list.swapPositions(7, 10);
146     return 0;
147 }

```

Answer 3

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  struct Node {
5      char data;
6      Node *next;
7  };
8  class LinkedList {
9      private:
10         Node *head;
11     public:
12         LinkedList() {
13             head = NULL;
14         }
15         void insert(char ch) {
16             Node *newNode = new Node;
17             newNode->data = ch;
18             newNode->next = NULL;
19             if (head == NULL) {
20                 head = newNode;
21                 return;
22             }
23             Node *temp = head;
24             while (temp->next != NULL)
25                 temp = temp->next;

```

```

26     temp->next = newNode;
27 }
28 void display() {
29     Node *temp = head;
30     while (temp != NULL) {
31         cout << temp->data << " ";
32         temp = temp->next;
33     }
34     cout << endl;
35 }
36 Node* getHead() {
37     return head;
38 }
39 void removeSequence(char a, char b, char c) {
40     Node *prev = NULL;
41     Node *curr = head;
42     while (curr && curr->next && curr->next->next) {
43         if (curr->data == a &&
44             curr->next->data == b &&
45             curr->next->next->data == c) {
46             Node *first = curr;
47             Node *second = curr->next;
48             Node *third = second->next;
49             Node *after = third->next;
50             if (prev == NULL)
51                 head = after;
52             else
53                 prev->next = after;
54             delete first;
55             delete second;
56             delete third;
57             return;
58         }
59         prev = curr;
60         curr = curr->next;
61     }
62 }
63 };
64 int main() {
65     LinkedList list1, list2;
66     cout << "Enter 10 characters for first linked list:\n";
67     for (int i = 0; i < 10; i++) {
68         char ch;
69         cin >> ch;
70         list1.insert(ch);
71     }
72     cout << "Enter 5 characters for second linked list:\n";
73     for (int i = 0; i < 5; i++) {
74         char ch;
75         cin >> ch;
76         list2.insert(ch);
77     }
78     cout << "\nFirst List : ";
79     list1.display();
80     cout << "Second List: ";
81     list2.display();
82     vector<char> v;
83     Node *temp = list2.getHead();
84     while (temp) {
85         v.push_back(temp->data);
86         temp = temp->next;
87     }
88     for (int i = 0; i <= 2; i++) {
89         list1.removeSequence(v[i], v[i + 1], v[i + 2]);
90     }
91     cout << "\nFirst list after removal:\n";
92     list1.display();
93     return 0;
94 }

```

Answer 4

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 class Node {
4     public:
5     int data;
6     Node* next;
7     Node(int value) {
8         data = value;
9         next = nullptr;
10    }
11 };
12 class LinkedList {
13     private:
14     Node* rear;
15     Node* head;
16     public:
17     LinkedList() {
18         head = nullptr;
19         rear = nullptr;
20    }
21     void insert(int value) {
22         Node* newNode = new Node(value);
23         if (head==NULL){
24             rear=newNode;
25             head=newNode;
26             rear->next=head;
27         }
28         else{
29             Node* curr=head;
30             while(curr->next!=head){
31                 curr=curr->next;
32             }
33             curr->next=newNode;
34             rear=newNode;
35             rear->next=head;
36         }
37    }
38     void display() {
39         if (head==NULL){
40             cout<<"Nothing to display\n";
41         }
42         if (head->next==head){
43             cout<<head->data<<"\n";
44             return;
45         }
46         cout<<head->data<<" ";
47         Node * curr=head->next;
48         while (curr!=head){
49             cout<<curr->data<<" ";
50             curr=curr->next;
51         }
52         cout<<"\n";
53    }
54     int cnt() {
55         Node * temp=head;
56         int c=1;
57         while(temp!=rear){
58             c++;
59             temp=temp->next;
60         }
61         return c;
62    }
63     void hasNegative() {
64         if (head == NULL) {
65             cout << "List is empty.\n";
66             return;
67         }
68         Node* curr = head;
```



```

69         do {
70             if (curr->data < 0) {
71                 cout << "Negative value found: " << curr->data <<
                        endl;
72                 return;
73             }
74             curr = curr->next;
75         } while (curr != head);
76         cout << "No negative values found.\n";
77     }
78     void countGreaterThan15() {
79         if (head == NULL) {
80             cout << "List is empty.\n";
81             return;
82         }
83         int count = 0;
84         Node* curr = head;
85         do {
86             if (curr->data > 15)
87                 count++;
88             curr = curr->next;
89         } while (curr != head);
90         cout << "Number of nodes with value greater than 15: " <<
                        count << endl;
91     }
92     void deleteNode(int value) {
93         Node *curr = head;
94         Node *prev = rear;
95         while (curr->data != value) {
96             prev = curr;
97             curr = curr->next;
98         }
99         if (head == rear) {
100             delete head;
101             head = rear = NULL;
102         }
103         else if (curr == head) {
104             head = head->next;
105             rear->next = head;
106             delete curr;
107         }
108         else if (curr == rear) {
109             prev->next = head;
110             rear = prev;
111             delete curr;
112         }
113         else {
114             prev->next = curr->next;
115             delete curr;
116         }
117         cout << "Element deleted successfully.\n";
118     }
119     void updateValue(int oldValue, int newValue) {
120         Node* curr = head;
121         while (curr->data != oldValue)
122             curr = curr->next;
123         curr->data = newValue;
124         cout << "Element updated successfully.\n";
125     }
126     void insertn(int a,int b,int c){
127         if(a==b+1){
128             insert(c);
129             return;
130         }
131         if(a==1){
132             Node* newNode = new Node(c);
133             Node* temp=rear;
134             temp->next=newNode;
135             temp->next=head;
136             head=temp;
137             return;

```

```

138     }
139     Node* curr=head;
140     for (int i=1;i<a-1;i++){
141         curr=curr->next;
142     }
143     Node* newNode = new Node(c);
144     Node* temp=curr->next;
145     curr->next=newNode;
146     curr=curr->next;
147     curr->next=temp;
148 }
149 };
150 int main(){
151     LinkedList list;
152     list.insert(1);
153     list.insert(2);
154     list.insert(3);
155     list.insert(4);
156     list.insert(5);
157     list.display();
158     cout<<"Numbers of nodes are:"<<list.cnt()<<"\n";
159     list.insertn(4,list.cnt(),10);
160     list.display();
161     list.hasNegative();
162     list.updateValue(10, 16);
163     list.countGreaterThan15();
164     list.deleteNode(3);
165     return 0;
166 }

```

Answer 5

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  class Node{
4      public:
5      int data;
6      Node* prev;
7      Node* next;
8      Node(int value){
9          data=value;
10         next=NULL;
11         prev=NULL;
12     }
13 };
14 class LinkedList{
15     private:
16     Node* head;
17     public:
18     LinkedList(){
19         head=NULL;
20     }
21     void insert(int value){
22         Node* newNode=new Node(value);
23         if(head==NULL){
24             head=newNode;
25             head->prev=NULL;
26             head->next=NULL;
27             return;
28         }
29         Node* curr=head;
30         while(curr->next!=NULL){
31             curr=curr->next;
32         }
33         curr->next=newNode;
34         newNode->prev=curr;
35     }
36     void display(){

```

```

37         if(head==NULL){
38             return;
39         }
40         Node* curr=head;
41         while(curr!=NULL){
42             cout<<curr->data<<" ";
43             curr=curr->next;
44         }
45         cout<<"\n";
46     }
47     void divisible(int m){
48         if(head==NULL){
49             return;
50         }
51         Node* curr=head;
52         while(curr!=NULL){
53             if((curr->data)%m==0){
54                 cout<<curr->data<<" is divisible by "<<m<<"\n";
55             }
56             curr=curr->next;
57         }
58     }
59     void deleteGreater(int x){
60         Node* curr = head;
61         while(curr != NULL){
62             if(curr->data > x){
63                 Node* temp = curr;
64                 if(curr == head){
65                     head = curr->next;
66                     if(head != NULL)
67                         head->prev = NULL;
68                 }
69                 else{
70                     curr->prev->next = curr->next;
71                     if(curr->next != NULL)
72                         curr->next->prev = curr->prev;
73                 }
74                 curr = curr->next;
75                 delete temp;
76             }
77             else{
78                 curr = curr->next;
79             }
80         }
81     }
82     int countBetweenDuplicates(){
83         Node* first = head;
84         while(first != NULL){
85             Node* curr = first->next;
86             int count = 0;
87             while(curr != NULL){
88                 if(curr->data == first->data){
89                     cout << "Number of elements between "
90                         << first->data << " = " << count << "\n";
91                     return count;
92                 }
93                 count++;
94                 curr = curr->next;
95             }
96             first = first->next;
97         }
98         cout << "No duplicate values found\n";
99         return 0;
100     }
101 };
102 int main() {
103     LinkedList list;
104     list.insert(1);
105     list.insert(2);
106     list.insert(3);

```

```

107     list.insert(4);
108     list.insert(5);
109     list.display();
110     list.divisible(3);
111     list.deleteGreater(3);
112     list.display();
113     list.countBetweenDuplicates();
114     return 0;
115 }

```

Answer 6

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  class Node{
4      public:
5      int data;
6      Node* prev;
7      Node* next;
8      Node(int value){
9          data=value;
10         next=NULL;
11         prev=NULL;
12     }
13 };
14 class LinkedList{
15     private:
16     Node* head;
17     public:
18     LinkedList(){
19         head=NULL;
20     }
21     void insert(int value){
22         Node* newNode=new Node(value);
23         if(head==NULL){
24             head=newNode;
25             return;
26         }
27         Node* curr=head;
28         while(curr->next!=NULL){
29             curr=curr->next;
30         }
31         curr->next=newNode;
32         newNode->prev=curr;
33     }
34     void display(){
35         Node* curr=head;
36         while(curr!=NULL){
37             cout<<curr->data<<" ";
38             curr=curr->next;
39         }
40         cout<<"\n";
41     }
42     void ExtremeSwap(){
43         if(head == NULL || head->next == NULL)
44             return;
45         Node* left = head;
46         Node* right = head;
47         while(right->next != NULL){
48             right = right->next;
49         }
50         while(left != right && left->prev != right){
51             swap(left->data, right->data);
52             left = left->next;
53             right = right->prev;

```

```

54     }
55 }
56 };
57 int main(){
58     LinkedList list;
59     list.insert(1);
60     list.insert(2);
61     list.insert(3);
62     list.insert(4);
63     list.insert(5);
64     list.insert(6);
65     list.insert(7);
66     list.insert(8);
67     cout<<"Original: ";
68     list.display();
69     list.ExtremeSwap();
70     cout<<"After 1st call: ";
71     list.display();
72     list.ExtremeSwap();
73     cout<<"After 2nd call: ";
74     list.display();
75     list.ExtremeSwap();
76     cout<<"After 3rd call: ";
77     list.display();
78     list.ExtremeSwap();
79     cout<<"After 4th call: ";
80     list.display();
81     return 0;
82 }

```

Answer 7

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  class Node {
4      public:
5      int coefficient;
6      int power;
7      Node* next;
8      Node(int c, int p) {
9          coefficient = c;
10         power = p;
11         next = NULL;
12     }
13 };
14 class Polynomial {
15     private:
16     Node* head;
17     public:
18     Polynomial() {
19         head = NULL;
20     }
21     void insert(int coefficient, int power) {
22         Node* newNode = new Node(coefficient, power);
23         if(head == NULL) {
24             head = newNode;
25             return;
26         }
27         Node* curr = head;
28         while(curr->next != NULL) {
29             curr = curr->next;
30         }
31         curr->next = newNode;
32     }
33     void display() {

```

```

34     Node* curr = head;
35     while(curr != NULL) {
36         cout << curr->coefficient << "x^" << curr->power;
37         if(curr->next != NULL)
38             cout << " + ";
39         curr = curr->next;
40     }
41     cout << endl;
42 }
43 Polynomial add(Polynomial& p) {
44     Polynomial result;
45     Node* a = head;
46     Node* b = p.head;
47     while(a != NULL && b != NULL) {
48         if(a->power == b->power) {
49             result.insert(a->coefficient + b->coefficient, a
50                             ->power);
51             a = a->next;
52             b = b->next;
53         }
54         else if(a->power > b->power) {
55             result.insert(a->coefficient, a->power);
56             a = a->next;
57         }
58         else {
59             result.insert(b->coefficient, b->power);
60             b = b->next;
61         }
62     }
63     while(a != NULL) {
64         result.insert(a->coefficient, a->power);
65         a = a->next;
66     }
67     while(b != NULL) {
68         result.insert(b->coefficient, b->power);
69         b = b->next;
70     }
71     return result;
72 };
73 int main() {
74     Polynomial p1, p2, result;
75     // 5x^3 + 4x^2 + 2
76     p1.insert(5, 3);
77     p1.insert(4, 2);
78     p1.insert(2, 0);
79     // 3x^3 + 2x^2 + 7x + 1
80     p2.insert(3, 3);
81     p2.insert(2, 2);
82     p2.insert(7, 1);
83     p2.insert(1, 0);
84     cout << "Polynomial 1: ";
85     p1.display();
86     cout << "Polynomial 2: ";
87     p2.display();
88     result = p1.add(p2);
89     cout << "Sum: ";
90     result.display();
91     return 0;
92 }

```

Answer 8

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  class Node {
4      public:
5      int coefficient;
6      int power;
7      Node* next;
8      Node(int c, int p) {
9          coefficient = c;
10         power = p;
11         next = NULL;
12     }
13 };
14 class Polynomial {
15     private:
16     Node* head;
17     public:
18     Polynomial() {
19         head = NULL;
20     }
21     void insert(int coefficient, int power) {
22         Node* newNode = new Node(coefficient, power);
23         if(head == NULL) {
24             head = newNode;
25             return;
26         }
27         Node* curr = head;
28         while(curr != NULL) {
29             if(curr->power == power) {
30                 curr->coefficient += coefficient;
31                 delete newNode;
32                 return;
33             }
34             curr = curr->next;
35         }
36         curr = head;
37         while(curr->next != NULL) {
38             curr = curr->next;
39         }
40         curr->next = newNode;
41     }
42     void display() {
43         Node* curr = head;
44         while(curr != NULL) {
45             cout << curr->coefficient << "x^" << curr->power;
46             if(curr->next != NULL)
47                 cout << " + ";
48             curr = curr->next;
49         }
50         cout << endl;
51     }
52     Polynomial multiply(Polynomial& p) {
53         Polynomial result;
54         Node* a = head;
55         while(a != NULL) {
56             Node* b = p.head;
57             while(b != NULL) {
58                 int coefficient = a->coefficient * b->coefficient
59                 ;
60                 int power = a->power + b->power;
61                 result.insert(coefficient, power);
62                 b = b->next;
63             }
64             a = a->next;
65         }
66         return result;
67     };
68     int main() {
69         Polynomial p1, p2, result;

```

```

70 // 2x^2 + 3x + 1
71 p1.insert(2, 2);
72 p1.insert(3, 1);
73 p1.insert(1, 0);
74 // 4x + 5
75 p2.insert(4, 1);
76 p2.insert(5, 0);
77 cout << "Polynomial 1: ";
78 p1.display();
79 cout << "Polynomial 2: ";
80 p2.display();
81 result = p1.multiply(p2);
82 cout << "Product: ";
83 result.display();
84 return 0;
85 }

```

Answer 9

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 class Node {
4     public:
5     int data;
6     Node* next;
7     Node(int value) {
8         data = value;
9         next = NULL;
10    }
11 };
12 class LinkedList {
13     private:
14     Node* head;
15     public:
16     LinkedList() {
17         head = NULL;
18     }
19     void insert(int value) {
20         Node* newNode = new Node(value);
21         if(head == NULL) {
22             head = newNode;
23             return;
24         }
25         Node* curr = head;
26         while(curr->next != NULL) {
27             curr = curr->next;
28         }
29         curr->next = newNode;
30     }
31     void rotateLeft(int k) {
32         if(head == NULL || head->next == NULL)
33             return;
34         int n = 1;
35         Node* last = head;
36         while(last->next != NULL) {
37             last = last->next;
38             n++;
39         }
40         k = k % n;
41         if(k == 0)
42             return;
43         Node* curr = head;
44         for(int i = 1; i < k; i++) {
45             curr = curr->next;
46         }
47         Node* newHead = curr->next;
48         last->next = head;

```



```

49         curr->next = NULL;
50         head = newHead;
51     }
52     void display() {
53         Node* curr = head;
54         while(curr != NULL) {
55             cout << curr->data << " ";
56             curr = curr->next;
57         }
58         cout << endl;
59     }
60 };
61 int main() {
62     LinkedList list;
63     list.insert(10);
64     list.insert(20);
65     list.insert(30);
66     list.insert(40);
67     list.insert(50);
68     list.insert(60);
69     cout << "Original list: ";
70     list.display();
71     list.rotateLeft(2);
72     cout << "After rotation: ";
73     list.display();
74     return 0;
75 }

```

Answer 10

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  class Node{
4      public:
5          int data;
6          Node* next;
7          Node(int value){
8              data = value;
9              next = NULL;
10         }
11 };
12 class LinkedList{
13     private:
14         Node* head;
15     public:
16         LinkedList(){
17             head = NULL;
18         }
19         void insert(int value){
20             Node* newNode = new Node(value);
21             if(head == NULL){
22                 head = newNode;
23                 return;
24             }
25             Node* curr = head;
26             while(curr->next != NULL){
27                 curr = curr->next;
28             }
29             curr->next = newNode;
30         }
31         void removeDuplicates(){
32             if(head == NULL)
33                 return;
34             Node* curr = head;
35             while(curr->next != NULL){
36                 if(curr->data == curr->next->data){
37                     Node* temp = curr->next;

```

```

38         curr->next = temp->next;
39         delete temp;
40     }
41     else{
42         curr = curr->next;
43     }
44 }
45 }
46 void display(){
47     Node* curr = head;
48     while(curr != NULL){
49         cout << curr->data << " ";
50         curr = curr->next;
51     }
52     cout << endl;
53 }
54 };
55 int main(){
56     LinkedList list;
57     list.insert(1);
58     list.insert(1);
59     list.insert(2);
60     list.insert(3);
61     list.insert(3);
62     list.insert(3);
63     list.insert(5);
64     list.insert(5);
65     list.insert(8);
66     cout << "Before: ";
67     list.display();
68     list.removeDuplicates();
69     cout << "After: ";
70     list.display();
71     return 0;
72 }

```